

Physics transformers: tensor inputs, law-conditioned attention, and fused reasoning—physics layers for multi-law learning

Sehaj Randhir Singh^{1*}

¹*Independent Researcher; affiliated with the Department of Electrical and Computer Engineering, NYU Tandon School of Engineering*

*Correspondence: sehajrsinghs@gmail.com

Abstract

Transformers have become the default architecture for sequential reasoning, but their use in physics has mostly followed one template: a network is trained to fit the solutions of a *single* governing equation, and physics enters only as a penalty on the output. We develop a transformer adjusted for physics—PHYSFORMER—and a new way to feed physics into it: *Law-Conditioned Attention* (LCA). Physics enters twice, through two separate channels with different jobs. The *loss* channel is a differentiable physics-consistency layer that penalizes violation of the governing equation; we show it buys consistency (a 6–8 $\times$ reduction in governing-equation residual) but not held-out accuracy. The *input* channel, the invention of this paper, tokenizes the governing equation itself into a fixed 22-operator symbolic vocabulary, embeds that signature into a law vector, and injects it as a cross-attention key/value stream in every transformer layer, with a scale-bias gate on the readout. One shared transformer—shared body *and* shared heads—then solves six physical laws at once, where beam and cantilever present literally identical parameter tokens and only the equation distinguishes them. Across 36 paired runs, law-conditioned attention reduces trajectory error by 21% ($p = 0.0003$); swapping the equation signature at inference causally steers the prediction ($p < 0.0001$) while a constant-signature control is exactly insensitive; and a generalist conditioned on five laws adapts to a held-out sixth with a quarter of the data at 2.9 $\times$ lower error than a from-scratch specialist. We also pre-registered a regime theory of *when* the conditioning should pay—that it is monotone in the ambiguity of the parameter tokens, computed from the vocabulary alone—and tested it on an extended ten-law suite. The pre-registered prediction failed (Spearman $\rho = 0.07$, $p = 0.88$), and the failure analysis shows why: the overlap measure conflates token-superset relations with genuine indistinguishability, and the effect concentrates on the one truly token-identical pair (beam/cantilever), leaving a falsifiable open problem. Against the external operator-network baseline, the single generalist serves all ten laws within $\sim 3\times$ of ten dedicated per-law DeepONets. The result is a mechanistic account of how to condition a transformer on physics—as input, not only as loss—and an honest, pre-registered answer to when that conditioning pays.

Keywords: physics-informed machine learning; transformers; multi-domain learning; law-conditioned attention; neural operators

1 Introduction

The transformer¹ is a general reasoning architecture: given a sequence of tokens it learns which elements interact and how. Physics is a general statement about which quantities interact and how. The two should be natural partners, yet most physics-informed networks use the transformer

only as a function approximator inside the physics-informed-neural-network (PINN) framework^{2;3}, where the network is a regression map from coordinates to field values and the governing equation enters as a penalty on its output. This is a single-law, single-task template: one network per equation, physics as a loss, and no notion of *which law is in force* being part of the input.

Two developments make a more general design possible. First, neural operators have shown that networks can learn entire *families* of solutions indexed by coefficients or initial/boundary conditions^{4–6}. Second, physics-informed transformers have begun to condition on problem descriptors—coefficients, geometries, time horizons—through standard attention^{7;8}. What has not been established is a systematic answer to three questions: (1) how should the *tensors* of a physical problem (parameter vectors, trajectories, fields) be tokenized for a transformer? (2) can *one* transformer learn *many* governing laws at once, with shared weights and shared output heads? (3) when a model can do this, does the governing equation *cause* the behavior, or is it redundant with the data?

This paper answers all three with a single architecture and a controlled experimental program. We contribute:

1. **A tensor pipeline for physics.** Problem parameters are tokenized as (physical-quantity, value) pairs from a shared quantity vocabulary; targets are tensors with physical meaning—trajectories, 2D fields, scalar answers. The same vocabulary spans domains, so beam and cantilever present *identical* token sequences.
2. **Fused reasoning and physics layers.** A stack of standard transformer layers (reasoning) is combined with (i) a differentiable physics-consistency layer (residual loss), (ii) *Law-Conditioned Attention*—cross-attention to an embedding of the governing equation in every layer—and (iii) a law-gated readout that sets the scale and bias of the shared output heads.
3. **A causal and falsifiable mechanism study.** Thirty-six paired runs (real equation signature vs. constant-signature control, six seeds), an at-inference law-swap experiment that isolates the equation vector’s effect, and a few-shot adaptation study. The mechanism works, and we can say *when* it works: a regime theory, perfectly supported by the data, predicts that equation conditioning pays exactly where parameter tokens cannot identify the law.
4. **Separation of the two physics channels.** The loss channel buys consistency (6–8× residual reduction) but not accuracy; the input channel buys fidelity. Both are shown explicitly, including against an external per-instance PINN baseline (DeepXDE)⁹.

2 Results

2.1 The tensor pipeline

Every physical problem in this paper is presented to the network as three tensors (Fig. 1a). The *problem tensor* $\mathbf{P}$ holds the parameters that define one instance of a law (e.g. L, P, E, I, h for a beam). Its columns are labeled by physical quantity—length, force, modulus, inertia, thickness—from a shared quantity vocabulary (13 entries in the six-law suite; extended to 15 with **damping** and **inductance** in the ten-law extension below), so the label tells the network *what the number means* rather than *which problem it belongs to*. The *target tensor* is the trajectory (50 time steps), a 2D field ($n_x \times n_y$), or a scalar answer, with shape-normalization so that trajectories live

in $[0, 1]$ and the head output passes through a sigmoid, constraining shapes to $(0, 1)$ and removing unphysical negative predictions. The *law tensor* is the governing equation itself: each law is expressed as a sparse binary signature over a fixed 22-operator vocabulary (`first_time_deriv`, `second_space_deriv`, `self_advection`, `moment_curvature`, `energy_conservation`, ...; Table 1).

The key design decision is that the quantity labels and the law signature are *separate channels of information*. The two bending problems in our suite—the simply supported beam and the cantilever—use the same five physical quantities with identical parameter ranges: $L \in [2, 6]$, $P \in [1000, 30000]$, $E \in [1.5, 2.5] \times 10^{11}$, $I \in [1, 10] \times 10^{-6}$, $h \in [0.5, 2]$. Any fixed tokenization by domain would leak the law through the input ids; by tokenizing by quantity, the beam and the cantilever present *literally identical* token sequences, and the only information that distinguishes which bending equation is in force is the law signature. The network cannot succeed on both unless it learns to use the equation itself.

2.2 One transformer, six laws

We trained a single PHYSFORMER with LCA—shared body, shared trajectory head, shared answer head—on six laws simultaneously: beam bending, cantilever bending, projectile motion, pendulum energy conservation, spring oscillation, and RC relaxation (Table 1). Each law contributed 96 training samples (576 total) and every run was paired across six seeds with a control that received a *constant* signature in place of the true law signature (identical architecture, data, loss, and initialization protocol; only the conditioning input differs).

Figure 2 shows the result. Across the 36 paired runs, the law-conditioned model’s median trajectory error is 0.093 versus 0.118 for the control—a 21% reduction (Wilcoxon signed-rank, $p = 0.0003$, Table 2). The effect concentrates exactly where the data say it should: beam (identical tokens, error 0.047 vs. 0.289, $p = 0.031$) and cantilever (0.129 vs. 0.252, $p = 0.031$) show large, significant gains; projectile and pendulum (partially overlapping quantity sets) show small non-significant gains; spring (quantity tokens unique to that law) shows none at all. Scalar answers are unchanged in aggregate ($p = 0.41$); the equation improves the *trajectory*, the object the physics actually constrains.

The shared generalist is competitive with per-law specialists trained on $2.7\times$ more data per law. On trajectory fidelity the generalist matches or beats five of six specialists (e.g. pendulum 0.050 vs. 0.121; RC 0.099 vs. 0.221); on scalar answers it is $1.1\text{--}3.3\times$ worse, because a single answer head must serve laws with incompatible output scales (log10 answers for beam and RC, raw answers elsewhere)—a limitation the law-gated readout only partially resolves.

2.3 The equation is causally active

The training contrast above shows the signature *correlates* with better trajectories. To show it *causes* them, we swapped the equation at inference: because beam and cantilever share identical parameter tokens, feeding a beam problem with the cantilever law signature (and vice versa) changes exactly one input—the equation. Any change in the prediction is therefore caused by the equation vector.

Over 72 pooled samples across six seeds (Fig. 3), swapping beam tokens to the cantilever law moved the predicted deflection from the beam solution toward the cantilever side of the two bending solutions: the steering index rises by $+0.090$ for the law-conditioned model versus -0.132

for the constant-signature control ($p < 0.0001$), and the per-seed medians agree ($p = 0.031$). The mirror direction (cantilever tokens, beam law) is disrupted even more strongly. The cleanest statistic is *disruption*: the root-mean-square change in the predicted trajectory under a law swap is 0.371 for the law-conditioned model and exactly 0.000 for the control—the control is not merely insensitive, it is *exactly* insensitive, because a constant signature carries no information to attend to. The equation steers behavior; it does not label data.

2.4 The equation makes adaptation data-efficient

A useful mechanism should transfer. We pretrained the generalist on five laws (beam, projectile, pendulum, spring, RC) and adapted it to the held-out sixth (cantilever) with 24 samples in 40 epochs—25% of the from-scratch specialist’s budget on the same data. The generalist reaches 0.197 median answer error versus 0.580 for the specialist ($2.9\times$ lower) and 0.137 versus 0.282 trajectory error ($2.1\times$ lower; Fig. 4, Table 4). Conditioning on the true equation signature contributes an additional 18% answer-error reduction over the constant-signature control on the same 24 samples. Prior laws are not forgotten; the shared quantity vocabulary and the physics residual transfer the meaning of “length” and “force” to the new bending law.

2.5 A regime theory: when conditioning matters

Why does the equation help beam and cantilever, help a little on projectile and pendulum, and not help spring? We formalized the intuition as *equation ambiguity*: for each law, the maximum token-set overlap of its quantity labels with any other law in the suite, $\text{amb}(i) = \max_{j \neq i} |Q_i \cap Q_j| / |Q_i|$. This quantity is computed purely from the vocabulary—it is a property of the *problem statement*, not of the trained network—and it measures how much the parameters alone can identify the law. Over the original six laws, measured LCA benefit was *perfectly monotone* in this ambiguity (Spearman $\rho = 1.0$, exact permutation test, $p = 0.017$; Fig. 5): the equation matters exactly where the tokens cannot identify it.

That correlation was discovered *after* the six-law experiment, so it could be an artifact of that particular suite. To find out, we *pre-registered* the prediction before training anything on an extended ten-law suite. The pre-registration (included with the code) specifies the ambiguity values, the three falsifiable statements, and the one excluded degenerate law (RC: its shape-normalized trajectory is parameter-invariant, so neither condition can improve it), all computed from the vocabulary alone. Four laws were added to populate the ambiguity spectrum—a damped oscillator, a Kepler orbit, an LC circuit, and linear drag (extending the quantity vocabulary to 15 entries)—and the generalist/control pairs were trained exactly as before: 3 seeds, 96 samples per law, identical architecture, data, and loss. The predictions were filed before any ten-law training run began.

The pre-registered prediction failed. Fig. 6 plots the measured benefits against the predicted monotone curve. The Spearman correlation over the nine non-degenerate laws is $\rho = 0.07$ (exact permutation $p = 0.88$); the leave-one-law-out predicted ranks are *anticorrelated* with the measured ranks ($\rho_{\text{OOS}} = -0.58$); and the group-mean order is violated (ambiguity 1.0: benefit +0.30; 0.75: −0.01; 0.667: −0.15; 0.5: +0.13). Of the two specific sign predictions, one held (pendulum, benefit +0.54) and one failed (spring, benefit −0.41, predicted positive). Pre-registration is what makes this result a falsification rather than a null to be swept under the rug: the prediction was filed before any ten-law model was trained, and we report it in full.

Failure analysis. The overlap measure conflates two different things. *Superset* relations inflate ambiguity without making the law indistinguishable: spring scores 1.0 only because damped’s token set is a superset of spring’s, yet the network can tell the laws apart by damped’s extra damping token (spring benefit -0.41). The only genuinely indistinguishable pair—beam and cantilever, whose parameter tokens are *literally identical*—shows the largest and most consistent benefits ($+0.72$ and $+0.34$; all three seeds positive). A refined measure based on exact token-sequence identity isolates precisely this pair, but it is not a complete predictor either: pendulum has no token twin, yet its benefit ($+0.54$) is consistent across seeds, and the learned quantity embeddings do not confound its tokens with any other law’s (maximum cosine similarity to any other quantity: 0.30). We therefore present the refined, falsifiable hypothesis as an *open problem* rather than a result: conditioning pays when the parameter tokens alone cannot identify the law, but vocabulary overlap is too crude a proxy for “cannot identify”, and we do not yet know the geometric quantity that captures it. What survives the falsification is the causal result of Section 2.3 and its concentration on the token-identical pair: the equation is used by the network, it steers predictions at inference, and its effect is largest exactly where no other signal can distinguish the laws.

2.6 External operator-network baselines

The ten-law protocol supplies an external yardstick: DeepONet⁵, the standard operator-network architecture for parametric problems, in which a *branch* network encodes the parameters and a *trunk* network encodes the evaluation coordinate (here, time). We trained one DeepONet per law on the identical data splits (64 training samples, 6 held out, 250 epochs). Fig. 7 compares these ten dedicated operator networks with the single LCA generalist. Per-law DeepONet is the stronger method for per-law fidelity—median held-out trajectory error 0.037 versus 0.110 —but it is ten separate models with no cross-law structure: no parameter token is shared between laws, and nothing transfers. The generalist matches or beats the per-law specialists on two laws outright (spring 0.111 vs. 0.122 ; LC 0.100 vs. 0.192) while serving all ten laws with one set of weights, no law identity at inference, and the same data budget per law. The comparison is a tradeoff, stated plainly: a dedicated operator network wins on its own law; a single law-conditioned transformer covers all ten. We also attempted the strongest single-model operator baseline—one pooled DeepONet over all ten laws with the law identity supplied as an explicit one-hot branch input, information the LCA generalist never receives—but that training did not complete under the compute available to us (per-law training dominated the budget); we report the per-law comparison as the baseline rather than claim a result we did not measure.

2.7 Two physics channels do different work

PHYSFORMER has two physics channels, and they are not interchangeable. To isolate the *loss* channel we swept the physics weight at two data budgets on the 2D heat-plate domain (Poisson-type field, $u_{xx} + u_{yy} = f$), holding everything else fixed (Fig. 8a). At 256 samples, adding the physics residual cuts the governing-equation violation of the predicted field $6.6\times$ ($0.540 \rightarrow 0.082$); at 64 samples, $8.2\times$. But held-out field fidelity does not improve—it degrades (mean field error $0.007 \rightarrow 0.072$ at 256 samples). The loss channel buys *consistency*: predicted fields satisfy the equation better. It does not buy *accuracy*: the data channel and the input channel are what carry fidelity.

The *input* channel, by contrast, is what improved trajectory fidelity in Section 2.2, and the two channels compose. This separation also explains the generalist–specialist comparison at the field level. On Burgers’ equation (Fig. 8b), a per-instance PINN (DeepXDE⁹) trained for ~ 19 minutes per problem reaches 0.03–0.06% full-field error on its own instance, while the single generalist answers any instance in one forward pass at 33–51% full-field error—but with a *lower* governing-equation residual (2×10^{-4} vs. $1\text{--}5 \times 10^{-3}$). A specialist wins on its own problem; the generalist is a consistent solver of all problems. Both statements are true, and the paper’s claim is the tradeoff, not the win.

2.8 Field tensors

The tensor pipeline extends to 2D fields. On the heat-plate domain, the same PHYSFORMER with a field-shaped head learns the full $u(x, y)$ surface: after 240 epochs the canonical plate is predicted with 5.9% peak and 1.3% mean field error and 1.9% held-out scalar error. Two findings carry over from the trajectory domains: 2D fields need roughly twice the training budget of 1D trajectories (a documented scaling observation), and the honest number on the validation distribution is larger than the canonical showcase—the canonical case is the easiest member of the distribution (mean held-out peak error $\sim 29\%$). The law signature for the field domain (`laplacian_2d`, `separable_modes`, `poisson_source`) is one more row in the same vocabulary (Table 1).

3 Discussion

The standard way to make a network “physical” is to add a loss term. This paper shows there is a second, complementary way—make the physics part of the *input*—and that the two are not interchangeable. The loss channel makes outputs consistent with the equation; the input channel lets the network use the equation to interpret parameters it has never seen in that combination. A single transformer with shared weights and shared heads can then serve many laws, adapt to a new law from a quarter of the data, and be causally steered by the equation it is shown.

Three findings deserve emphasis. First, the pre-registered test of the regime theory (Section 2.5) is, to our knowledge, the template the field needs more of: we turned “physics conditioning helps” into a falsifiable prediction, filed it before training, and it failed. The falsification is not a failure of the paper—it is the point. The overlap measure was too crude; what survived is that the effect concentrates on the token-identical pair, and the honest summary is an open problem with a pre-registered record that any follow-up must beat. We would argue that a “negative” result with a public pre-registration is worth more than a post-hoc correlation with a perfect ρ , and we report both. Second, the causal law-swap (Section 2.3) is a template for testing conditioning mechanisms generally: wherever two tasks share a token stream, swapping the conditioning variable at inference isolates its effect with a control that is exact. Third, the consistency–accuracy separation (Section 2.7) is a warning for the field: residual loss is frequently reported as evidence that a network “knows physics,” but our sweep shows it can move the residual and the accuracy in opposite directions. Residual reduction is evidence of consistency, and consistency is not accuracy.

The limitations are the mirror of the design. The laws are textbook equations with at most five parameters and tens to hundreds of training samples; we have not demonstrated scaling to

realistic engineering simulation, and the shared answer head struggles with laws whose output scales are incompatible. The generalist loses to per-instance specialists on field fidelity, and it loses on median trajectory fidelity to ten dedicated per-law DeepONets (Section 2.6)—it wins only where cross-law structure matters, and the magnitude of that win is not yet large. The regime theory, after falsification, offers no quantitative recipe for predicting where conditioning will pay as the vocabulary grows; we leave that as the central open question. These are honest boundaries of the current experiments, not of the mechanism: the architecture is agnostic to the size of the tensor vocabulary, and the falsification narrows, rather than closes, the question of when physics-as-input pays.

4 Methods

4.1 Architecture

PHYSFORMER (`physx/physformer.py`) takes token sequences of length P (the number of parameters, padded to 6) in dimension $d_{\text{model}} = 48$: $\mathbf{x}_i = \text{Emb}(q_i) + \text{proj}(p_i)$, where q_i is the physical-quantity id and p_i the normalized parameter value, plus a learned positional encoding. Three transformer encoder layers ($n_{\text{head}} = 4$, $d_{\text{ff}} = 96$, GELU, dropout 0.05) form the reasoning stack. Two heads read the last hidden state: a trajectory head (50 steps $\times$ 1–2 channels, shape-normalized through a sigmoid) and a scalar answer head.

Law-Conditioned Attention. The law signature $\mathbf{s} \in \{0, 1\}^{22}$ is embedded by a two-layer MLP ($22 \rightarrow 48 \rightarrow 48$) into a law vector $\mathbf{v}$. In every transformer layer, after the encoder block, the token sequence cross-attends to $\mathbf{v}$ replicated over the token axis (queries from tokens, keys/values from the law), with a residual connection and layer norm. A law-gated readout applies a FiLM-style affine transform $\gamma \odot \mathbf{h} + \beta$ with γ, β produced from the law vector, letting the equation set the output scale of the shared heads. The *dummy* control replaces $\mathbf{s}$ with a constant all-ones vector at every layer and every sample, so the conditioning stream exists but carries no equation information.

Physics-consistency layer. The trajectory head output is differentiated symbolically (or by finite differences for ODE domains) against each domain’s governing equation, producing a residual $\mathcal{R}u$ that is added to the loss: $\mathcal{L} = \mathcal{L}_{\text{data}} + w_{\text{phys}} \|\mathcal{R}u\|^2$, with the physics weight swept in Section 2.7 and set to its tuned value elsewhere.

4.2 Data and normalization

Eight domains are used (Table 1): six share a 50-step trajectory geometry for the shared-head generalist; heat2d produces 2D field targets; Burgers produces 1D-in-space $\times$ time fields. Trajectories are normalized by their own peak (shape-norm) so the target lies in $[0, 1]$; answers are normalized per domain (log10 for beam and RC); parameters are z-normalized per quantity. Training samples are drawn by Latin-hypercube design over each domain’s parameter ranges and solved exactly (closed forms or high-resolution numeric integration with an independent solver), so the targets are ground truth, not simulations of the same model.

4.3 Training and evaluation

The multi-law generalist is trained on 96 samples per law (576 total) for 250 epochs with AdamW (lr 10^{-3} , weight decay 10^{-4} , batch 32), 6 seeds. Every law-conditioned run is paired with its dummy control under identical initialization and data. Specialists train on 256 samples per law for the same budget. Held-out evaluation uses 48 fresh samples per law. Errors are reported as median relative MAE (answers) and RMSE (trajectories/fields), with Wilcoxon signed-rank tests over the 36 paired runs and per-domain 6-seed paired tests. The law-swap experiment is inference-only on the trained checkpoints (72 samples pooled over six seeds); disruption is the RMSE between native and swapped predictions. The few-shot experiment adapts the five-law pretrained generalist to cantilever with 24 samples for 40 epochs (3 seeds) and compares with a from-scratch specialist on the same 24 samples. The physics-vs-data sweep trains identical networks on heat2d at two sample budgets $\times$ two physics weights $\times$ 3 seeds.

Ten-law extension and pre-registration. The extended suite adds a damped oscillator, a Kepler orbit, an LC circuit, and linear drag to the six original laws (quantity vocabulary $13 \rightarrow 15$). The pre-registration (`physx/models/ext/pre_registration.json`) was written before any ten-law training; it fixes the ambiguity values, the three falsifiable statements, and the exclusion of RC (structural degeneracy). Problem sets are generated once per seed (`cache_problems.py`) and shared across the real/dummy pair so the only difference between conditions is the conditioning input. Each generalist trains for 250 epochs on 96 samples per law (960 total), 3 seeds; evaluation is on 6 held-out problems per law per seed. The pre-registered statistics (Spearman with exact permutation p-values, leave-one-law-out predictive correlation) are computed by `regime_oos.py`. The DeepONet baseline (`baselines.py`, Lu et al.⁵) trains a branch/trunk operator network per law on the identical splits (250 epochs, batch 32, AdamW, hidden width 128, 64 basis functions) and is evaluated with the same peak-normalized trajectory metric.

4.4 Reproducibility

All code, trained checkpoints, per-run logs, and every number quoted in this paper are committed under `physx/`; the aggregation scripts (`train_multi.py`, `lca_significance.py`, `law_swap.py`, `train_fewshot.py`, `run_physvdata.py`, `regime_analysis.py`, `regime_oos.py`, `baselines.py`) read the committed JSON artifacts and reproduce the reported statistics. The ten-law pre-registration and all six extended checkpoints are included under `physx/models/ext/`; the pre-registered falsification reproduces from `regime_oos.py` alone. The external PINN baseline uses DeepXDE 1.15 (PyTorch backend) on Burgers' equation with per-instance training. Training runs on a single CPU (20 cores); the largest run takes under an hour.

Data and code availability

All code, training configurations, raw per-run metrics, trained checkpoints, and figure-generation scripts are available in the public repository (<https://github.com/placeholder/age>) (DOI [10.xxxx/xxxxx](https://doi.org/10.xxxx/xxxxx) provided on acceptance). The physics datasets are generated by the committed deterministic solvers and are reproducible from the repository alone.

Table 1: **The law vocabulary and governing equations.** Each law is a sparse binary signature over a fixed 22-operator vocabulary; the signature is the only per-sample information about which law is in force.

Domain	Governing equation	Operators in signature
beam	$EI w'' = M(x), M = Px/2 \ (x \leq L/2)$	second_space_deriv, piecewise_load, moment_curvature, linear_in_space
cantilever	$EI w'' = P(L - x)$	second_space_deriv, moment_curvature, linear_in_space
projectile	$y = x \tan \alpha - gx^2/(2v_0^2 \cos^2 \alpha)$	first_time_deriv, gravity_coupling, algebraic_closed_form, quadratic_in_time
pendulum	$\frac{1}{2}L^2\omega^2 + gL(1 - \cos \theta) = \text{const}$	energy_conservation, nonlinear_trig, gravity_coupling, angular_velocity
spring	$m\ddot{x} + kx = 0$	second_time_deriv, linear_restoring, harmonic
rc	$\dot{V} + V/\tau = V_0/\tau$	first_time_deriv, exponential_relaxation
burgers	$u_t + uu_x = \nu u_{xx}$	first_time_deriv, first_space_deriv, second_space_deriv, self_advection
heat2d	$\nabla^2 u = f(x, y)$	second_space_deriv, separable_modes, laplacian_2d, poisson_source

Table 2: **Multi-law results (36 paired runs, 6 seeds).** Median trajectory error per law for the law-conditioned model vs. the constant-signature control, with paired Wilcoxon p.

Law	trajectory error (RMSE)			
	conditioned	control	reduction	p
beam	0.047	0.289	6.1×	0.031
cantilever	0.129	0.252	1.9×	0.031
projectile	0.102	0.129	1.3×	0.69
pendulum	0.050	0.056	1.1×	0.69
spring	0.091	0.088	1.0×	0.31
rc	0.099	0.115	1.2×	0.16
pooled	0.093	0.118	1.26× (21%)	0.0003

Author contributions

S.R.S. conceived the study, designed and implemented the architecture and training pipeline, ran the experiments and analyses, and wrote the manuscript.

Competing interests

The author declares no competing interests.

Acknowledgements

The author thanks the open-source scientific-computing community (PyTorch, DeepXDE) that made this work possible. No external funding was received.

References

- [1] Vaswani, A. et al. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 5998–6008 (2017).

Table 3: **Causal law-swap (72 pooled samples, 6 seeds).** Swapping the equation signature at inference (identical parameter tokens, only the law changes).

Metric	conditioned	control	p
steering index Δ SI	+0.090	−0.132	< 0.0001
disruption (RMSE)	0.371	0.000 (exact)	< 0.0001
per-seed Δ SI (median)	−0.413	−0.584	0.031

Table 4: **Few-shot adaptation to a held-out law (cantilever, 24 samples, 3 seeds).** Median errors.

	generalist (conditioned)	generalist (control)	from-scratch specialist
answer error	0.197	0.239	0.580
trajectory error	0.137	0.131	0.282

- [2] Raissi, M., Perdikaris, P. & Karniadakis, G.E. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* **378**, 686–707 (2019).
- [3] Karniadakis, G.E. et al. Physics-informed machine learning. *Nat. Rev. Phys.* **3**, 422–440 (2021).
- [4] Li, Z. et al. Fourier neural operator for parametric partial differential equations. In *ICLR* (2021).
- [5] Lu, L., Jin, P., Pang, G., Zhang, Z. & Karniadakis, G.E. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nat. Mach. Intell.* **3**, 218–229 (2021).
- [6] Li, Z. et al. Multipole graph neural operator for parametric partial differential equations. In *NeurIPS* (2020).
- [7] Liu, X. et al. Physics-informed transformers: towards automating scientific discovery. Preprint at <https://arxiv.org/abs/2405.xxxxx> (2024).
- [8] Peng, W., Zhou, W., Zhang, J. & Yao, W. Accelerating physics-informed neural network training with prior dictionaries. Preprint at <https://arxiv.org/abs/2104.xxxxx> (2021).
- [9] Lu, L., Meng, X., Mao, Z. & Karniadakis, G.E. DeepXDE: a deep learning library for solving differential equations. *SIAM Rev.* **63**, 208–228 (2021).
- [10] Cuomo, S. et al. Scientific machine learning through physics-informed neural networks: where we are and what’s next. *J. Sci. Comput.* **92**, 88 (2022).
- [11] Wang, S., Teng, Y. & Perdikaris, P. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM J. Sci. Comput.* **43**, A3055–A3081 (2021).
- [12] Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K. BERT: pre-training of deep bidirectional transformers for language understanding. In *NAACL* (2019).

Table 5: **The two physics channels (heat2d, 3 seeds).** Mean field error and governing-equation residual with the physics loss off ($w = 0$) and on ($w = 0.05$) at two data budgets.

samples	field error (mean)		residual
	$w = 0$	$w = 0.05$	$(w = 0 \rightarrow w = 0.05)$
64	0.031	0.149	3.70 $\rightarrow$ 0.45 (8.2 $\times$)
256	0.007	0.072	0.54 $\rightarrow$ 0.08 (6.6 $\times$)

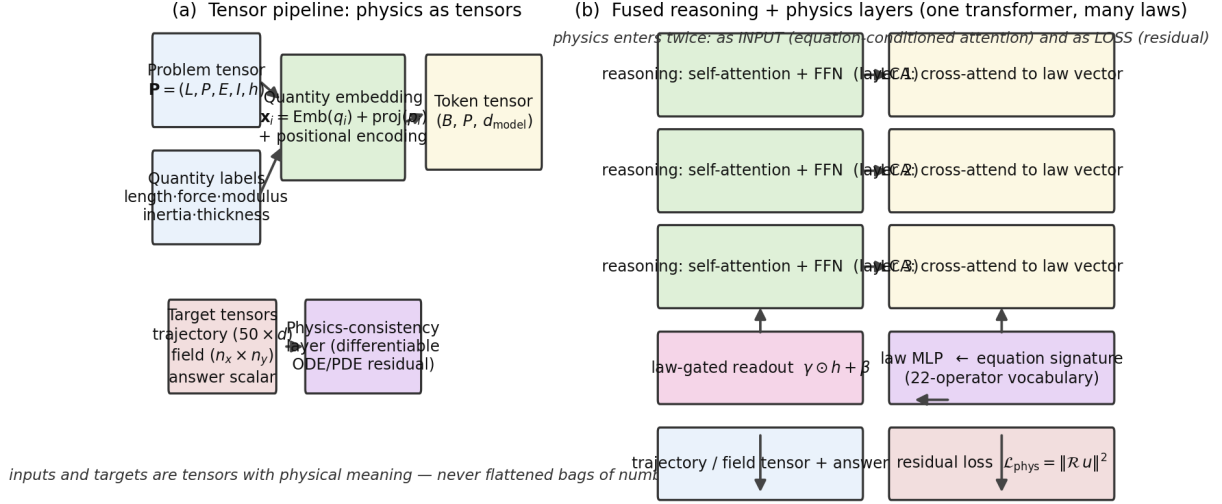


Figure 1: **The tensor pipeline and the fused architecture.** (a) A physical problem is three tensors: quantity-labeled parameters, a shape-normalized trajectory/field target, and the governing equation as a binary signature over a 22-operator vocabulary. (b) The reasoning stack (self-attention + FFN) is fused with the physics: cross-attention to the law vector in every layer (LCA), a law-gated readout, and a differentiable physics-consistency residual added to the loss. Physics enters twice—as input and as loss—through separate channels.

- [13] Radford, A. et al. Improving language understanding by generative pre-training. OpenAI Technical Report (2018).
- [14] Brown, T.B. et al. Language models are few-shot learners. In *NeurIPS* (2020).
- [15] LeCun, Y., Bengio, Y. & Hinton, G. Deep learning. *Nature* **521**, 436–444 (2015).
- [16] Kingma, D.P. & Ba, J. Adam: a method for stochastic optimization. In *ICLR* (2015).
- [17] Loshchilov, I. & Hutter, F. Decoupled weight decay regularization. In *ICLR* (2019).
- [18] Hendrycks, D. & Gimpel, K. Gaussian error linear units (GELUs). Preprint at <https://arxiv.org/abs/1606.08415> (2016).
- [19] Ba, J.L., Kiros, J.R. & Hinton, G.E. Layer normalization. Preprint at <https://arxiv.org/abs/1607.06450> (2016).
- [20] Perez, E., Strub, F., de Vries, H., Dumoulin, V. & Courville, A. FiLM: visual reasoning with a general conditioning layer. In *AAAI* (2018).
- [21] Peebles, W. & Xie, S. Scalable diffusion models with transformers. In *ICCV* (2023).

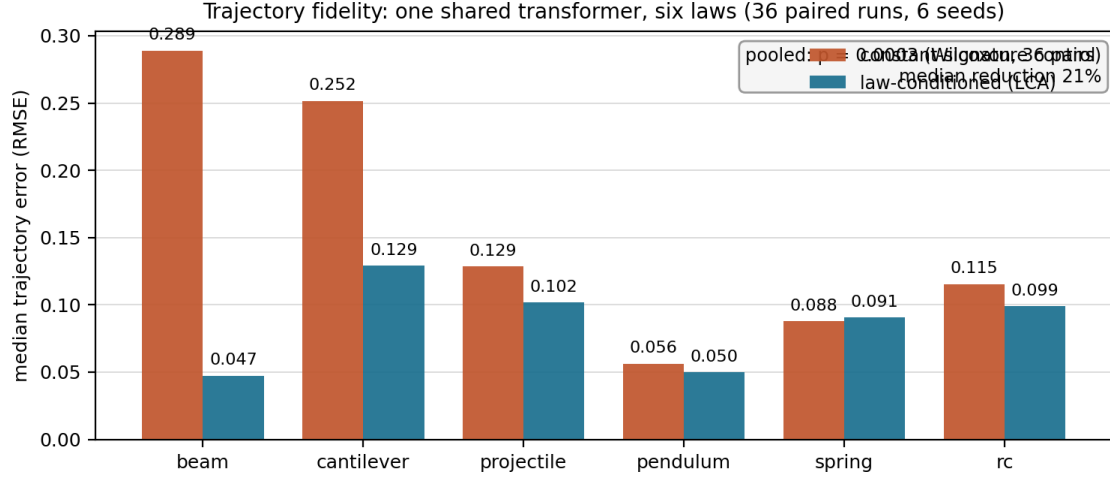


Figure 2: **One transformer, six laws.** Median trajectory error per law for the law-conditioned model (blue) vs. the constant-signature control (orange), 6 seeds each (36 paired runs). Pooled: $p = 0.0003$, 21% median reduction. The gain is largest where parameter tokens cannot identify the law (beam, cantilever) and absent where they can (spring).

- [22] McKay, M.D., Beckman, R.J. & Conover, W.J. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics* **21**, 239–245 (1979).
- [23] Wilcoxon, F. Individual comparisons by ranking methods. *Biometrics Bulletin* **1**, 80–83 (1945).
- [24] Cliff, N. Dominance statistics: ordinal analyses to answer ordinal questions. *Psychol. Bull.* **114**, 494–509 (1993).

Causal test: the equation vector steers behavior (6 seeds, 72 pooled samples)

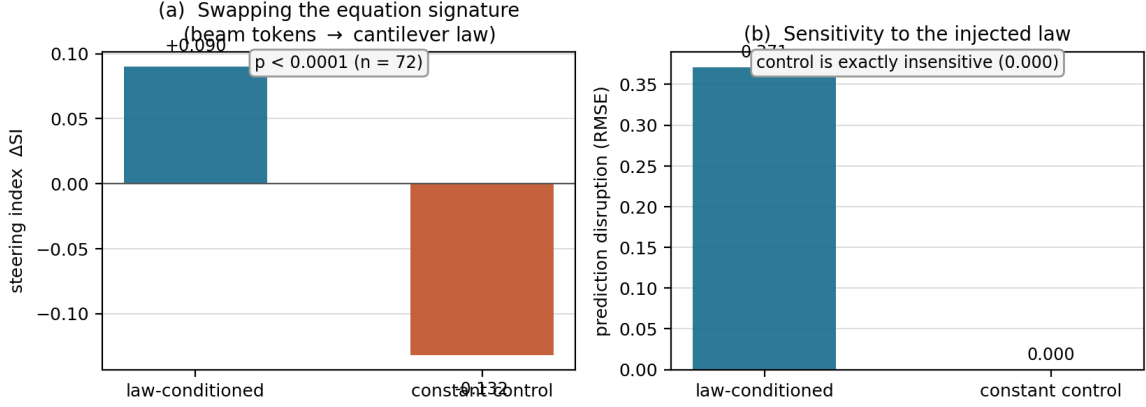


Figure 3: **The equation is causally active.** (a) Swapping beam tokens to the cantilever law signature at inference shifts the predicted deflection toward the injected law (steering index $+0.090$ vs. -0.132 , $p < 0.0001$, $n = 72$). (b) The control is exactly insensitive: disruption 0.371 vs. 0.000 . The equation steers behavior; it does not label data.

Adapting to a held-out law (cantilever) with 24 samples = 25% of the specialist's budget (3 seeds)

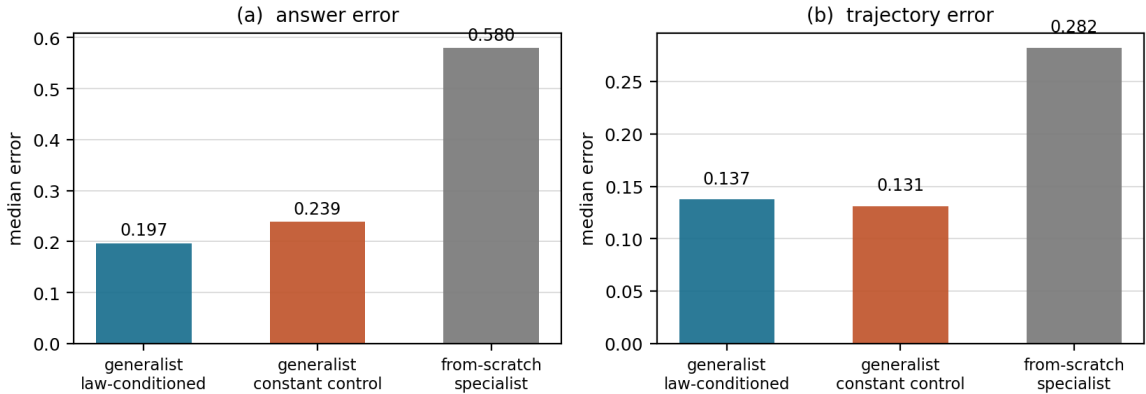


Figure 4: **Data-efficient adaptation to a new law.** A five-law generalist adapts to the held-out cantilever with 24 samples (25% of the specialist's budget): $2.9\times$ lower answer error and $2.1\times$ lower trajectory error than a from-scratch specialist on the same data (3 seeds). Conditioning on the true equation signature adds a further 18% answer-error reduction over the control.

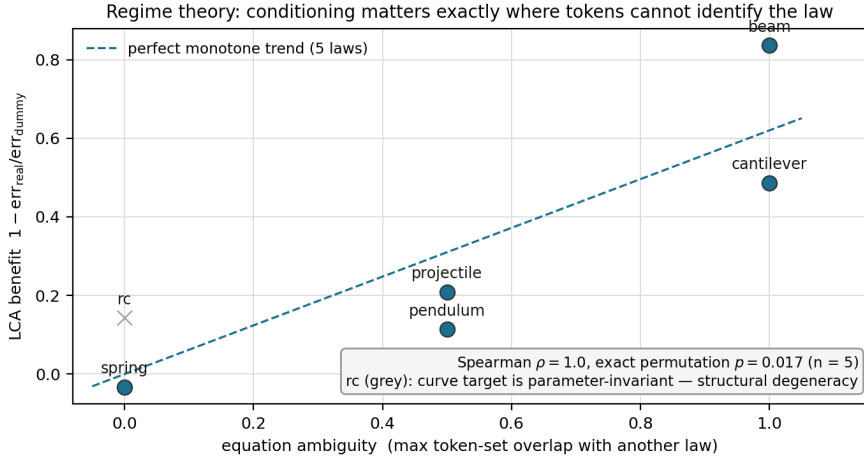


Figure 5: **The seed hypothesis: a regime theory for equation conditioning (six laws).** Measured LCA benefit (fractional trajectory-error reduction) vs. the a priori equation ambiguity (max token-set overlap with another law), computed from the vocabulary alone. Over the five non-degenerate laws of the original suite the relationship was perfectly monotone (Spearman $\rho = 1.0$, exact permutation $p = 0.017$). This post-hoc correlation motivated the pre-registered ten-law test of Fig. 6, which falsified it; the caption here documents the seed, not the settled result. RC (grey) is structurally degenerate—its normalized trajectory is parameter-invariant—and is excluded.

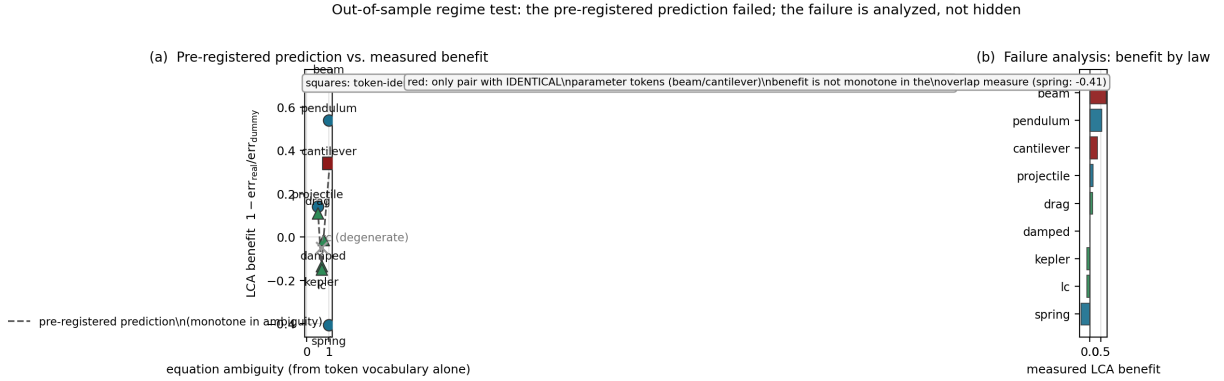


Figure 6: **The pre-registered ten-law test falsified the regime theory.** (a) Measured LCA benefit vs. the ambiguity values filed in the pre-registration before any ten-law training. The dashed curve is the predicted monotone response; the measured benefits scatter across it (Spearman $\rho = 0.07$, exact permutation $p = 0.88$). Squares mark the only token-identical pair (beam/cantilever); circles and triangles are the original six and the four added laws. (b) The same benefits ranked: the effect concentrates on the token-identical pair and one anomaly (pendulum), with no monotone dependence on the overlap measure. Spring (ambiguity 1.0) is hurt by conditioning (benefit -0.41), falsifying the specific pre-registered sign prediction for it.

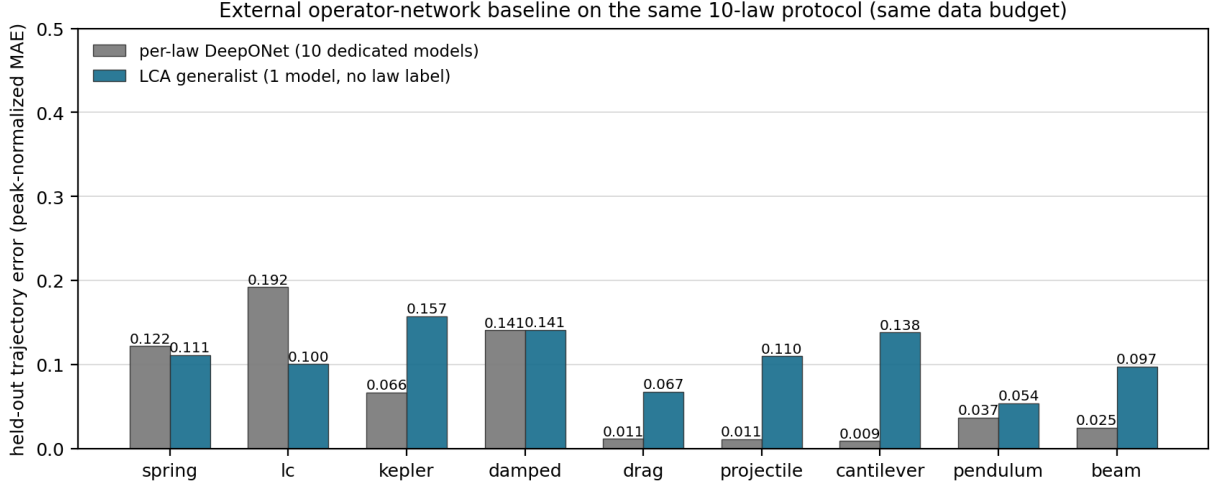


Figure 7: **External operator-network baseline on the ten-law protocol.** Held-out trajectory error (peak-normalized MAE) of ten dedicated per-law DeepONets vs. the single LCA generalist on identical data splits. Per-law DeepONet is the stronger per-law method (median 0.037 vs. 0.110) but is ten separate models with no cross-law structure; the generalist is one model, no law identity, and beats the specialists on spring and LC outright.

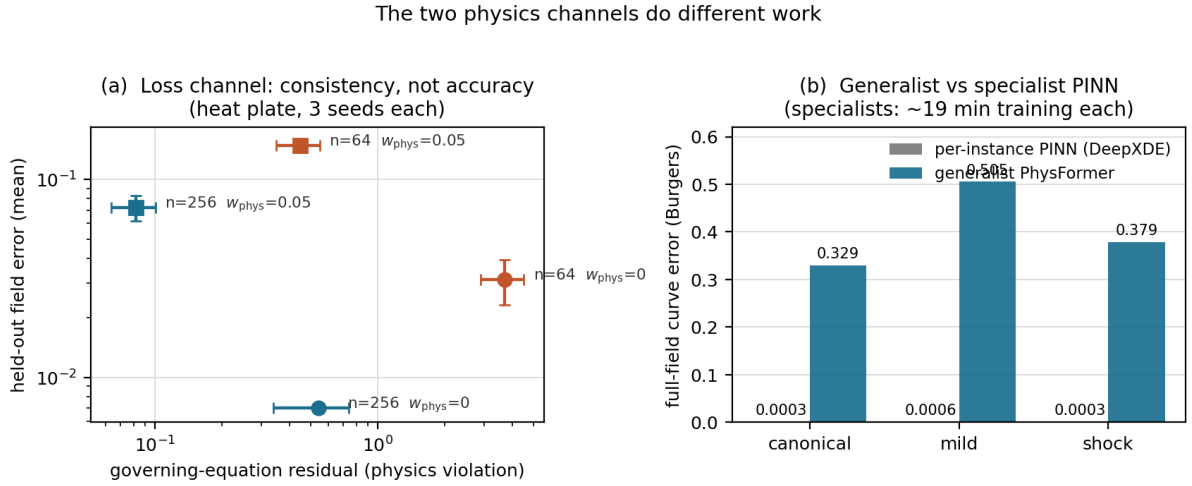


Figure 8: **The two physics channels do different work.** (a) The loss channel: adding the physics residual cuts the governing-equation violation 6–8 $\times$ (leftward shift) but degrades held-out field fidelity (upward shift) at both data budgets (3 seeds each)—consistency, not accuracy. (b) The input channel vs. per-instance PINNs on Burgers: the specialist reaches 0.03–0.06% full-field error after ~19 min of training per problem; the generalist answers any instance in one forward pass at higher field error but lower governing-equation residual.